

JAWAHARLAL NEHRU TECHNOLOGICAL UNIVERSITY

GURAJADA VIZIANAGARAM

VIZIANAGARAM – 533003

ANDHRA PRADESH

SOS SAFETY SYSTEM

By

SANAPALA PRASUNA

(Regd. No. 24VV1A1252)

Under the supervision of

MRS. MADHUMITHA CHANDA

MRS. M.GEETHA MADHURI YADAV

Department of Information Technology

Vizianagaram – 533003



JNTU-GURAJADA VIZIANAGARAM
COLLEGE OF ENGINEERING VIZIANAGARAM (A)
VIZIANAGARAM

Regd.No : 24VV1A1252

CERTIFICATE

This is to certify that this is a bonafide record of practical work done by **SANAPALA PRASUNA** of IIIrd B.Tech Ist Semester Class in User Interface Design using Flutter Lab during the year 2026-27.

No.of Tasks Completed and Certified: 11

Lecture In-Charge **Head of The Department**

Date:

DEPARTMENT OF INFORMATION TECHNOLOGY
JNTU-GURAJADA VIZIANAGARAM
COLLEGE OF ENGINEERING VIZIANAGARAM (A)
VIZIANAGARAM

Website:www.jntugvcev.edu.in

Subject Name: USER INTERFACE DESIGN USING FLUTTER Subject Code: R233112ES01

Academic Year: 2026 **Regulation: R23**

COURSE OUTCOMES

NBA Subject Code	Course Outcomes	
R233112ES01	CO1	Learns to Implement Flutter Widgets and Layouts.
	CO2	Understands Responsive UI Design and with Navigation in Flutter.
	CO3	Knowledge on Widgets and customize widgets for specific UI elements, Themes
	CO4	Understand to include animation apart from fetching data.

CO-PO Mapping

Mapping of Course Outcomes (COs) with Program Outcomes (POs)

Course Outcomes		Program Outcomes (POs)														
		PO 1	PO 2	PO 3	PO 4	PO 5	PO 6	PO 7	PO 8	PO 9	PO 10	PO 11	PO 12	PSO 1	PSO 2	PSO 3
R233112ES01	CO1	3	2	3	2	3	1	1	1	2	2	1	2	3	3	2
	CO2	3	3	3	2	3	1	2	2	2	3	2	2	3	3	2
	CO3	3	2	3	2	3	2	1	1	2	2	1	2	3	2	2
	CO4	3	3	3	3	3	2	1	2	3	3	2	3	3	3	3

Enter correlation levels 1,2 and 3 as defined below:

1: Slight (Low) 2: Moderate (Medium) 3: Substantial (High) If there is no correlation, put "-"

Signature of the Course Instructor

CONTENTS

S.No.	Date	Experiment Name	Page No.	Marks	Signature
1(a)		Installation of Flutter and Dart SDK	1		
1(b)		Dart Programming Basics	3		
2(a)		Exploring Flutter Widgets in SOS	4		
2(b)		Layouts Using Row, Column and Stack	6		
3(a)		Responsive User Interface	8		
3(b)		Media Queries and Breakpoints	10		
4(a)		Navigation Between Application Screens	12		
4(b)		Navigation Using Named Routes	14		
5(a)		Stateful and Stateless Widgets	16		
5(b)		State Management Using Provider	18		
6(a)		Creating Reusable Custom Widgets	20		
6(b)		Themes and Custom Styles	23		
7(a)		Creating Forms with Input Fields	26		
7(b)		Form Validation and Error Handling	28		
8(a)		Implementing UI Animations	31		
8(b)		Page Transition Animations	33		
9(a)		Connecting the Application to Firebase	36		
9(b)		Retrieving and Displaying Application Data	39		
10(a)		Unit Testing in Flutter	42		
10(b)		Debugging a Flutter Application	44		
11		Case Study: SOS Safety System	46		

EXPERIMENT 1(a)

INSTALLATION OF FLUTTER AND DART SDK

Aim:
To install Flutter and Dart SDK and configure the development environment for Flutter application development.

Description:
Flutter is an open-source UI toolkit used to build applications for mobile, web and desktop platforms from a single codebase. Dart is the programming language used by Flutter. The SOS Safety System was developed using Flutter and Dart.

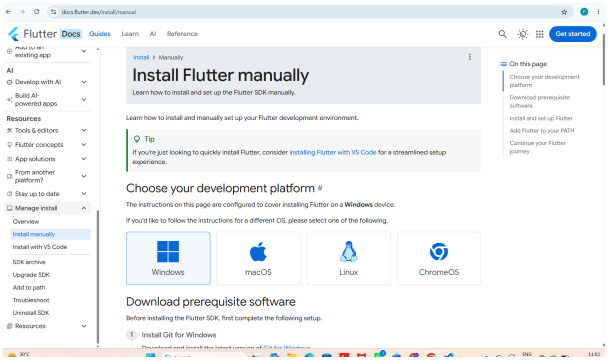
Requirements:

- Windows operating system.
- Flutter SDK and Dart SDK.
- Visual Studio Code or Android Studio.
- Android SDK and an emulator or Android device.

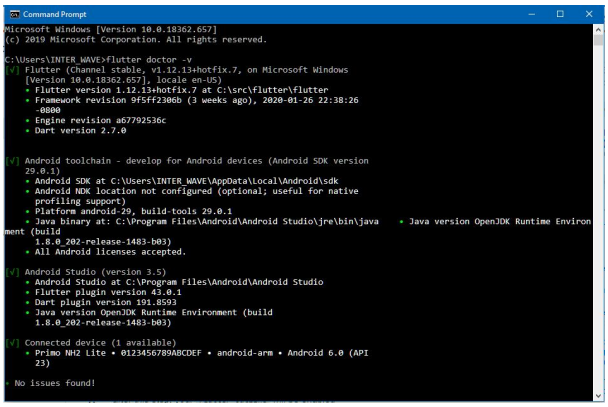
Procedure:

1. Download the Flutter SDK from `https://docs.flutter.dev/get-started/install`.
2. Extract the SDK into a suitable directory.
3. Add the Flutter `bin` directory to the system PATH.
4. Verify the installation using `flutter --version`.
5. Run `flutter doctor` and install the required tools.
6. Create or open the SOS Safety System Flutter project.

Flutter SDK Installation:



Flutter Doctor Output:



Output:

The Flutter development environment is configured and the SOS Safety System project can be opened and executed.

Result:

Thus, Flutter and Dart SDK were installed and configured for application development.

EXPERIMENT 1(b)

DART PROGRAMMING BASICS

Aim:

To understand Dart programming concepts used in the SOS Safety System.

Description:

Dart supports variables, data types, functions, classes, lists, maps and asynchronous programming. These concepts are used in the project to represent user profiles, emergency contacts, SOS records and location information.

Program:

```dart
class EmergencyContact {
  final String name;
  final String phone;

  const EmergencyContact({
    required this.name,
    required this.phone,
  });
}

void main() {
  final contacts = <EmergencyContact>[
    const EmergencyContact(name: 'Contact 1', phone: '0000000000'),
    const EmergencyContact(name: 'Contact 2', phone: '1111111111'),
  ];

  for (final contact in contacts) {
    print('${contact.name}: ${contact.phone}');
  }
}
```

Output:

The program displays the names and phone numbers stored in the emergency contact list.

Result:

Thus, basic Dart concepts were implemented using an emergency contact example.

EXPERIMENT 2(a)

EXPLORING FLUTTER WIDGETS IN SOS

Aim:

To study and implement basic Flutter widgets used in the SOS Safety System interface.

Description:

Widgets are the basic building blocks of Flutter applications. The project uses widgets such as Scaffold, AppBar, Text, Icon, Card, Column, ElevatedButton and ListTile to create application screens.

Program:

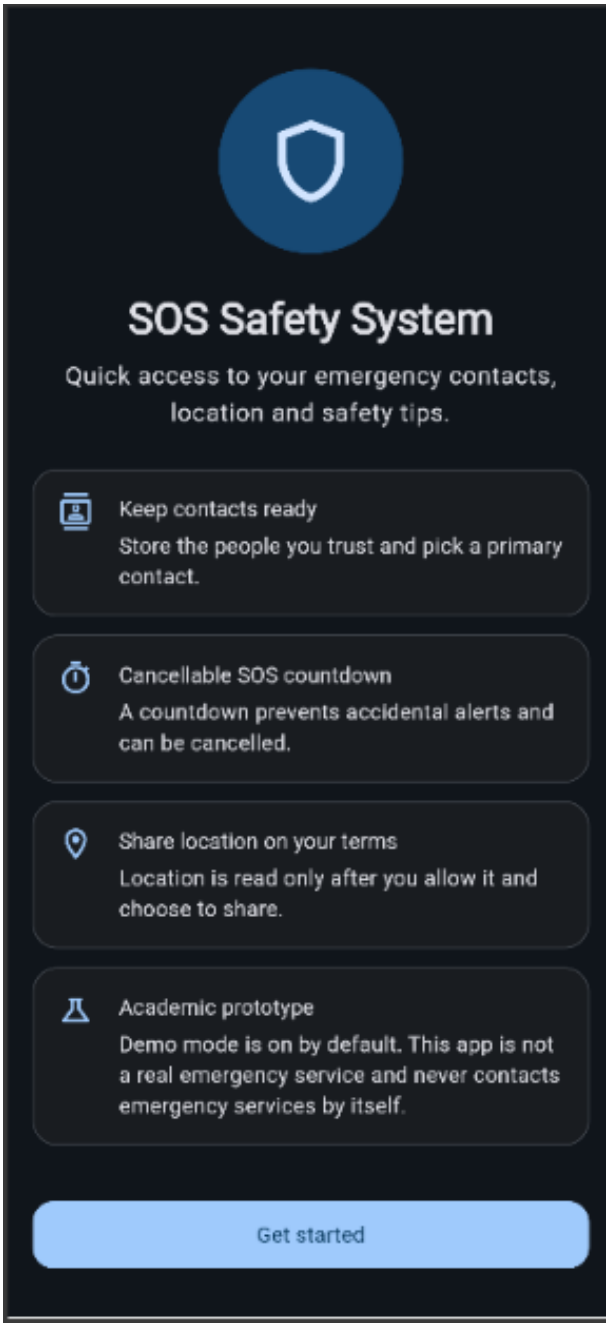
```dart
import 'package:flutter/material.dart';

class SafetyHomeDemo extends StatelessWidget {
  const SafetyHomeDemo({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('SOS Safety System')),
      body: Padding(
        padding: const EdgeInsets.all(16),
        child: Column(
          children: [
            const Icon(Icons.health_and_safety, size: 60),
            const Text('Your Safety Matters'),
            Card(
              child: ListTile(
                leading: const Icon(Icons.contacts),
                title: const Text('Emergency Contacts'),
                subtitle: const Text('Manage trusted contacts'),
              ),
            ),
            ElevatedButton(
              onPressed: () {},
              child: const Text('Open SOS'),
            ),
          ],
        ),
      ),
    );
  }
}
```

Output:

The interface displays an application bar, safety icon, emergency contact card and SOS button.



Result:

Thus, basic Flutter widgets were used to construct the SOS application interface.

EXPERIMENT 2(b)

LAYOUTS USING ROW, COLUMN AND STACK

Aim:

To arrange SOS application interface elements using Row, Column and Stack widgets.

Description:

Row arranges widgets horizontally, Column arranges them vertically and Stack places widgets on top of one another. These layouts help organize dashboard cards, icons and action buttons.

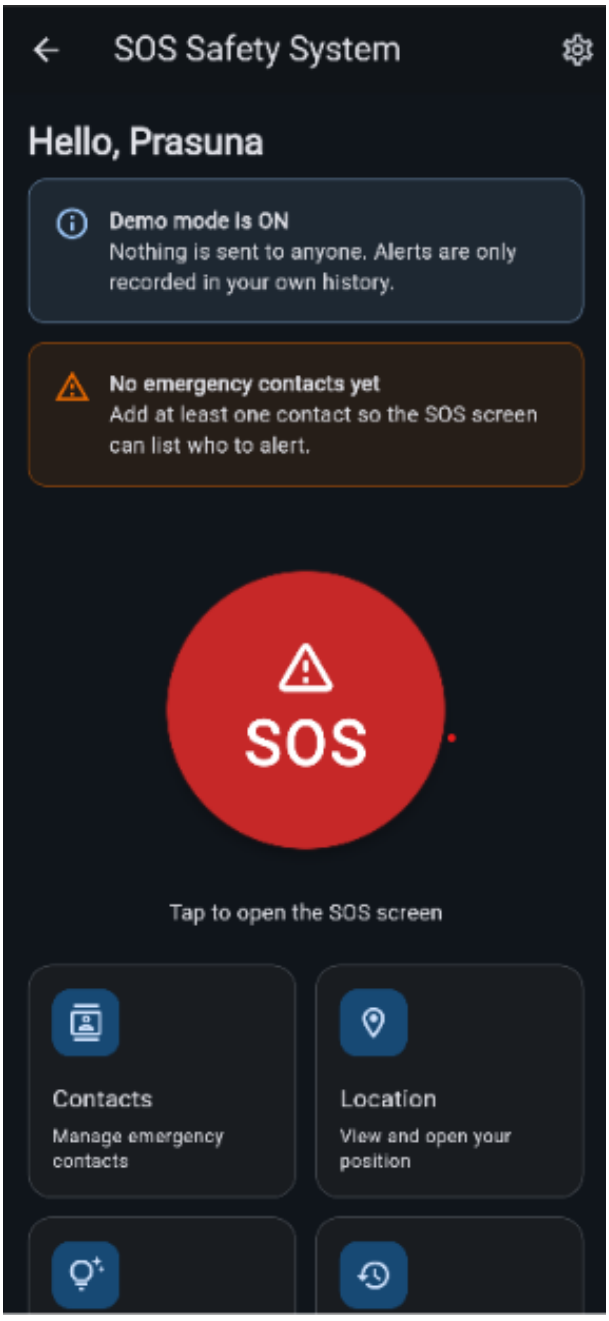
Program:

```
Column(
  crossAxisAlignment: CrossAxisAlignment.stretch,
  children: [
    Row(
      children: [
        Expanded(
          child: Card(
            child: ListTile(
              leading: Icon(Icons.contacts),
              title: Text('Contacts'),
            ),
          ),
        ),
        Expanded(
          child: Card(
            child: ListTile(
              leading: Icon(Icons.location_on),
              title: Text('Location'),
            ),
          ),
        ),
      ],
    ),
    const SizedBox(height: 16),
    Stack(
      alignment: Alignment.center,
      children: [
        Container(height: 110, color: Colors.red.shade50),
        const Icon(Icons.sos, size: 55, color: Colors.red),
      ],
    ),
  ],
```

```
33 )
```

Output:

The interface arranges feature cards horizontally and displays an SOS icon in a stacked layout.



Result:

Thus, Row, Column and Stack widgets were used to organize the application interface.

EXPERIMENT 3(a)

RESPONSIVE USER INTERFACE

Aim:

To create a responsive user interface that adjusts to different screen sizes.

Description:

A responsive interface adapts its layout to the available screen width. The SOS Safety System uses reusable responsive layout widgets to keep the dashboard and feature sections usable on different devices.

Program:

```dart
import 'package:flutter/material.dart';

class ResponsiveDemo extends StatelessWidget {
  const ResponsiveDemo({super.key});

  @override
  Widget build(BuildContext context) {
    return LayoutBuilder(
      builder: (context, constraints) {
        final columns = constraints.maxWidth >= 700 ? 2 : 1;

        return GridView.count(
          crossAxisCount: columns,
          padding: const EdgeInsets.all(16),
          children: const [
            Card(child: Center(child: Text('SOS'))),
            Card(child: Center(child: Text('Contacts'))),
            Card(child: Center(child: Text('Location'))),
            Card(child: Center(child: Text('Safety Tips'))),
          ],
        );
      },
    );
  }
}
```

Output:

The feature cards are arranged according to the available screen width.

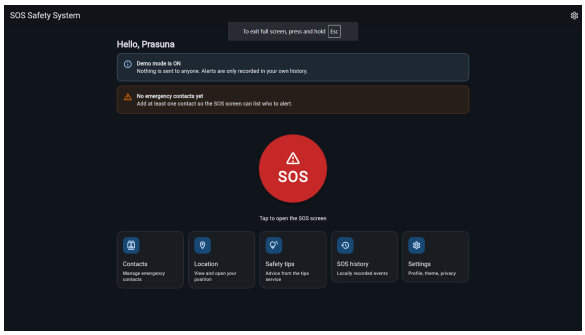


Figure 1: Desktop View

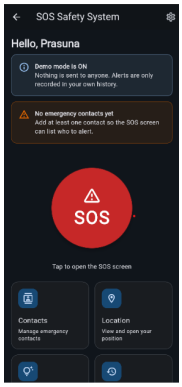


Figure 2: Mobile View

Result:

Thus, a responsive interface was implemented using Flutter layout constraints.

EXPERIMENT 3(b)

MEDIA QUERIES AND BREAKPOINTS

Aim:

To use screen width breakpoints to select suitable layouts.

Description:

MediaQuery provides information about the screen size. Breakpoints define width ranges for selecting layouts. The project also contains a breakpoint utility to support responsive presentation.

Program:

```dart
import 'package:flutter/material.dart';

class BreakpointDemo extends StatelessWidget {
  const BreakpointDemo({super.key});

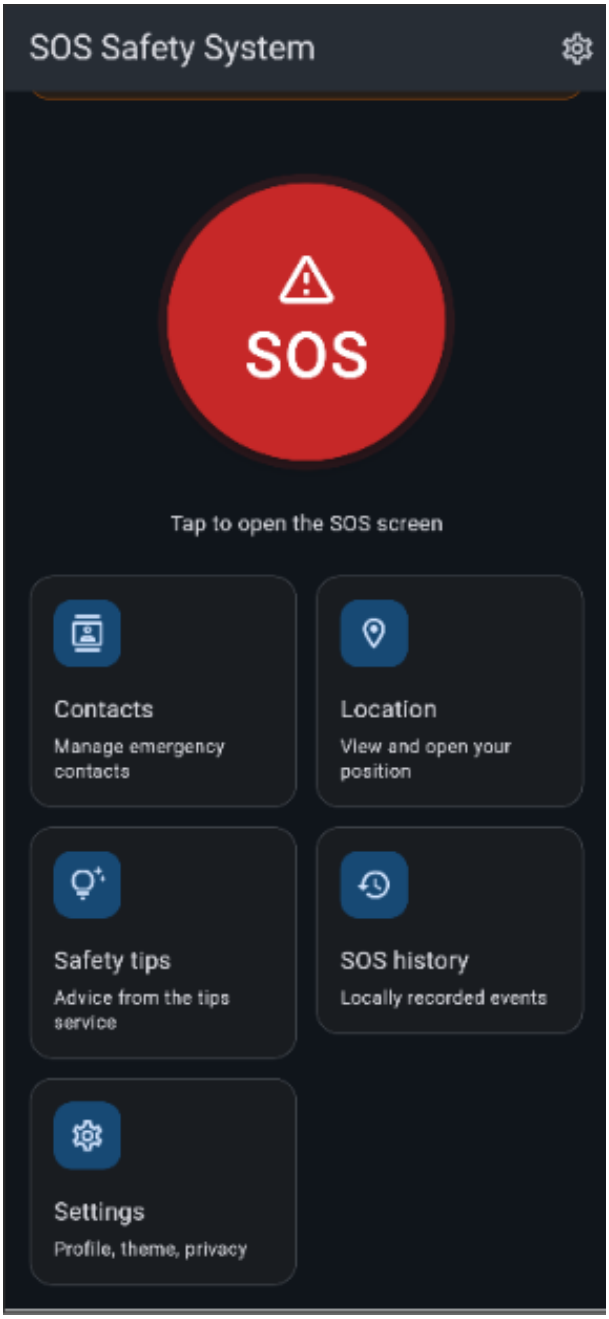
  @override
  Widget build(BuildContext context) {
    final width = MediaQuery.sizeOf(context).width;

    final deviceLayout = width < 600
        ? 'Mobile Layout'
        : width < 1000
            ? 'Tablet Layout'
            : 'Desktop Layout';

    return Scaffold(
      body: Center(child: Text(deviceLayout)),
    );
  }
}
```

Output:

The application selects a mobile, tablet or desktop layout according to the screen width.



Result:

Thus, MediaQuery and breakpoints were used to adapt the user interface.

EXPERIMENT 4(a)

NAVIGATION BETWEEN APPLICATION SCREENS

Aim:

To navigate between different screens of the SOS Safety System.

Description:

Flutter Navigator manages a stack of routes. The push method opens another screen and pop returns to the previous screen. The SOS application contains screens for home, contacts, location, tips, history and settings.

Program:

```dart
import 'package:flutter/material.dart';

void main() => runApp(const MaterialApp(home: Home()));

class Home extends StatelessWidget {
  const Home({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('SOS Safety System')),
      body: Center(
        child: ElevatedButton(
          child: const Text('Emergency Contacts'),
          onPressed: () {
            Navigator.push(
              context,
              MaterialPageRoute(
                builder: (context) => const ContactScreen(),
              ),
            );
          },
        ),
      ),
    );
  }
}

class ContactScreen extends StatelessWidget {
  const ContactScreen({super.key});

  @override
```

```
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Emergency Contacts')),
      body: Center(
        child: Column(
          mainAxisAlignment: MainAxisAlignment.center,
          children: [
            const Icon(Icons.contacts, size: 60),
            const Text('Manage Emergency Contacts'),
            ElevatedButton(
              onPressed: () => Navigator.pop(context),
              child: const Text('Go Back'),
            ),
          ],
        ),
      ),
    );
  }
}
```

Output:

The application navigates from the home screen to the Emergency Contacts screen using Navigator.

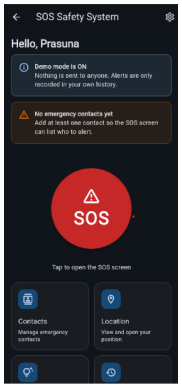


Figure 3: Home Screen

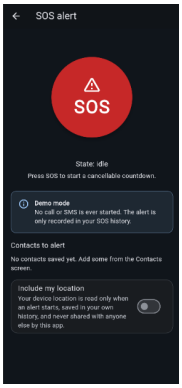


Figure 4: SOS Alert Screen

Result: Thus, screen navigation was implemented using Navigator.push() and Navigator.pop().

EXPERIMENT 4(b)

NAVIGATION USING NAMED ROUTES

Aim:

To implement named route navigation in the SOS Safety System.

Description:

Named routes identify screens using string constants. The project defines route names in the AppRoutes class and uses an AppRouter to generate the corresponding screens.

Program:

```dart
import 'package:flutter/material.dart';

void main() {
  runApp(const MaterialApp(
    debugShowCheckedModeBanner: false,
    initialRoute: '/',
    routes: {
      '/': (context) => const HomeScreen(),
      '/contacts': (context) => const ContactsScreen(),
    },
  ));
}

class HomeScreen extends StatelessWidget {
  const HomeScreen({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('SOS Safety System')),
      body: Center(
        child: ElevatedButton(
          onPressed: () {
            Navigator.pushNamed(context, '/contacts');
          },
          child: const Text('Emergency Contacts'),
        ),
      ),
    );
  }
}

class ContactsScreen extends StatelessWidget {
```

```
  const ContactsScreen({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Emergency Contacts')),
      body: Center(
        child: Column(
          mainAxisAlignment: MainAxisAlignment.center,
          children: [
            const Icon(Icons.contacts, size: 60),
            const Text('Manage Emergency Contacts'),
            ElevatedButton(
              onPressed: () => Navigator.pop(context),
              child: const Text('Go Back'),
            ),
          ],
        ),
      ),
    );
  }
}
```

Output:

The named route opens the Emergency Contacts screen when the corresponding button is clicked.

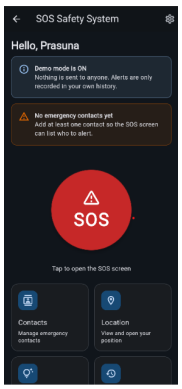


Figure 5: Home Screen

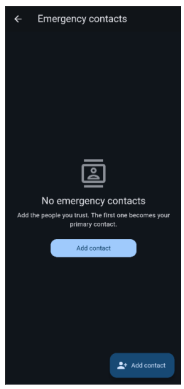


Figure 6: Emergency Contacts Screen

Result:

Thus, named route navigation was implemented using Navigator.pushNamed() successfully.

EXPERIMENT 5(a)

STATEFUL AND STATELESS WIDGETS

Aim:

To understand StatefulWidget and StatelessWidget in Flutter.

Description:

StatelessWidget is used when a widget does not manage changing internal state. StatefulWidget is used when the interface changes in response to user actions or application data. The SOS application uses both types of widgets.

Program:

```
class SafetyTitle extends StatelessWidget {
  const SafetyTitle({super.key});

  @override
  Widget build(BuildContext context) {
    return const Text('SOS Safety System');
  }
}

class CounterDemo extends StatefulWidget {
  const CounterDemo({super.key});

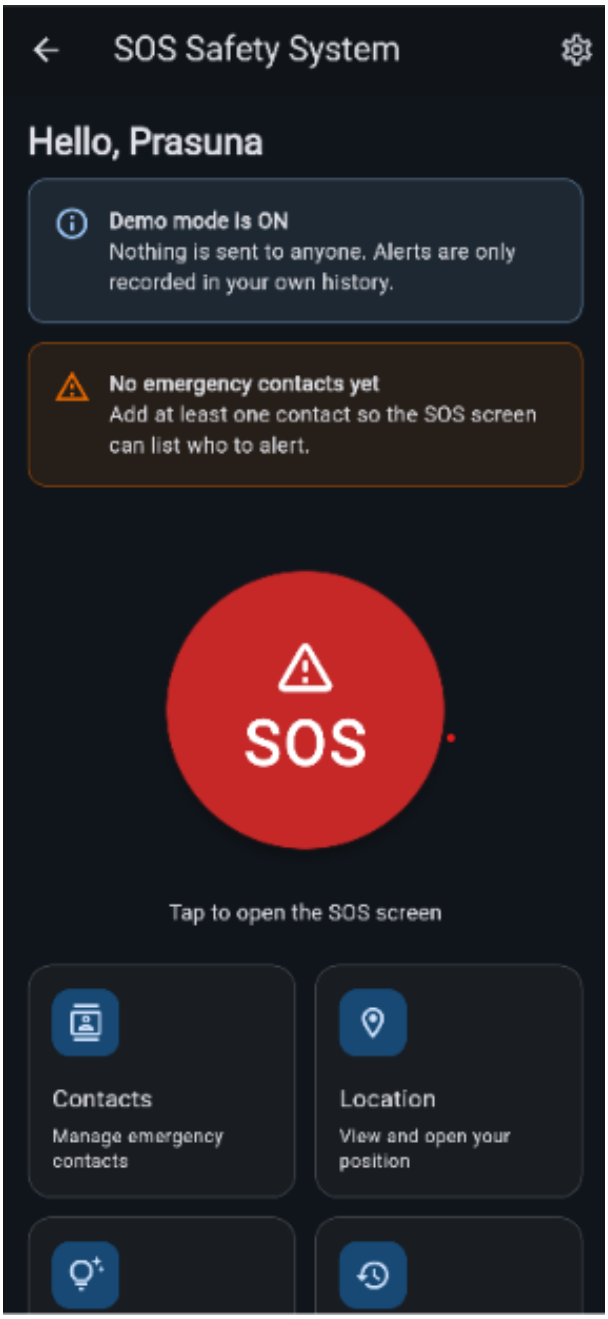
  @override
  State<CounterDemo> createState() => _CounterDemoState();
}

class _CounterDemoState extends State<CounterDemo> {
  int count = 0;

  @override
  Widget build(BuildContext context) {
    return ElevatedButton(
      onPressed: () => setState(() => count++),
      child: Text('Count: $count'),
    );
  }
}
```

Output:

The title remains unchanged, while the counter updates when the button is pressed.



Result:

Thus, StatelessWidget and StatefulWidget were demonstrated in Flutter.

EXPERIMENT 5(b)

STATE MANAGEMENT USING PROVIDER

Aim:

To manage application state using the Provider package.

Description:

Provider makes application data available to multiple widgets. The SOS Safety System uses ChangeNotifierProvider and ChangeNotifier-based providers to manage authentication, contacts, location, SOS events, history, settings and safety tips.

Program:

```dart
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';

class SafetyCounter extends ChangeNotifier {
  int count = 0;

  void increment() {
    count++;
    notifyListeners();
  }
}

void main() {
  runApp(
    ChangeNotifierProvider(
      create: (_) => SafetyCounter(),
      child: const MaterialApp(home: CounterPage()),
    ),
  );
}

class CounterPage extends StatelessWidget {
  const CounterPage({super.key});

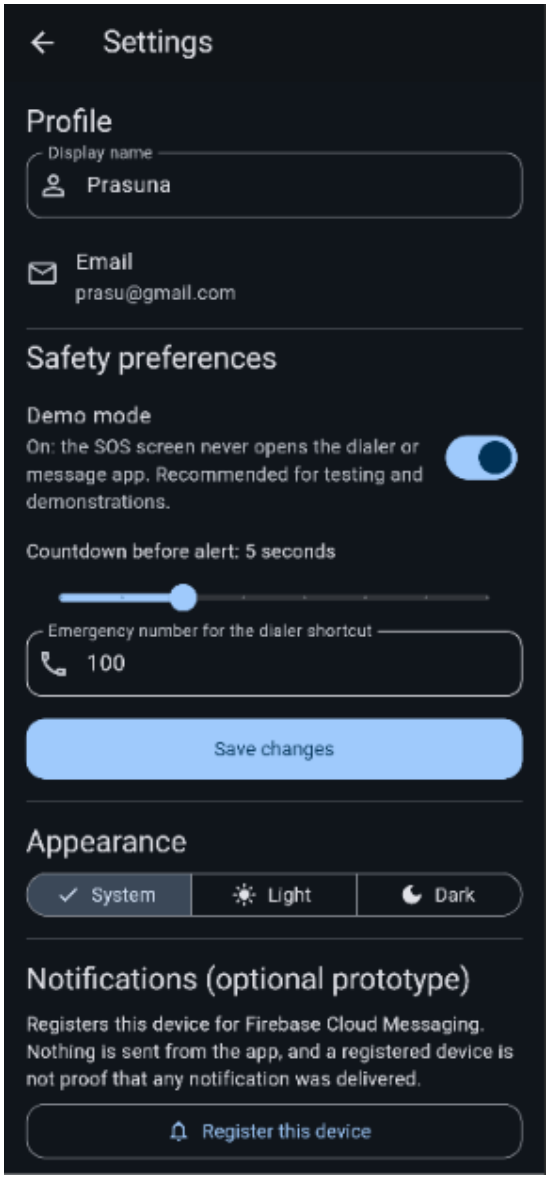
  @override
  Widget build(BuildContext context) {
    final state = context.watch<SafetyCounter>();

    return Scaffold(
      body: Center(
        child: Text('Count: ${state.count}'),
      ),
```

```
      floatingActionButton: FloatingActionButton(
        onPressed: () => context.read<SafetyCounter>().increment(),
        child: const Icon(Icons.add),
      ),
    );
  }
}
```

Output:

The displayed value changes when the state is updated and listeners are notified.



Result:

Thus, state management was demonstrated using Provider and ChangeNotifier.

EXPERIMENT 6(a)

CREATING REUSABLE CUSTOM WIDGETS

Aim:

To create reusable custom widgets for the SOS Safety System.

Description:

Custom widgets reduce code duplication and improve maintainability. The project includes reusable widgets such as SOSButton, ContactCard, FeatureTile, TipCard, StatusBanner and ResponsiveLayout.

Program:

```
import 'package:flutter/material.dart';

class SafetyCard extends StatelessWidget {
  final String title;
  final IconData icon;
  final Color color;

  const SafetyCard({
    super.key,
    required this.title,
    required this.icon,
    required this.color,
  });

  @override
  Widget build(BuildContext context) {
    return Card(
      child: Padding(
        padding: const EdgeInsets.all(16),
        child: Column(
          mainAxisSize: MainAxisSize.min,
          children: [
            Icon(icon, size: 35, color: color),
            const SizedBox(height: 10),
            Text(
              title,
              style: const TextStyle(
                fontWeight: FontWeight.bold,
              ),
            ),
          ],
        ),
```

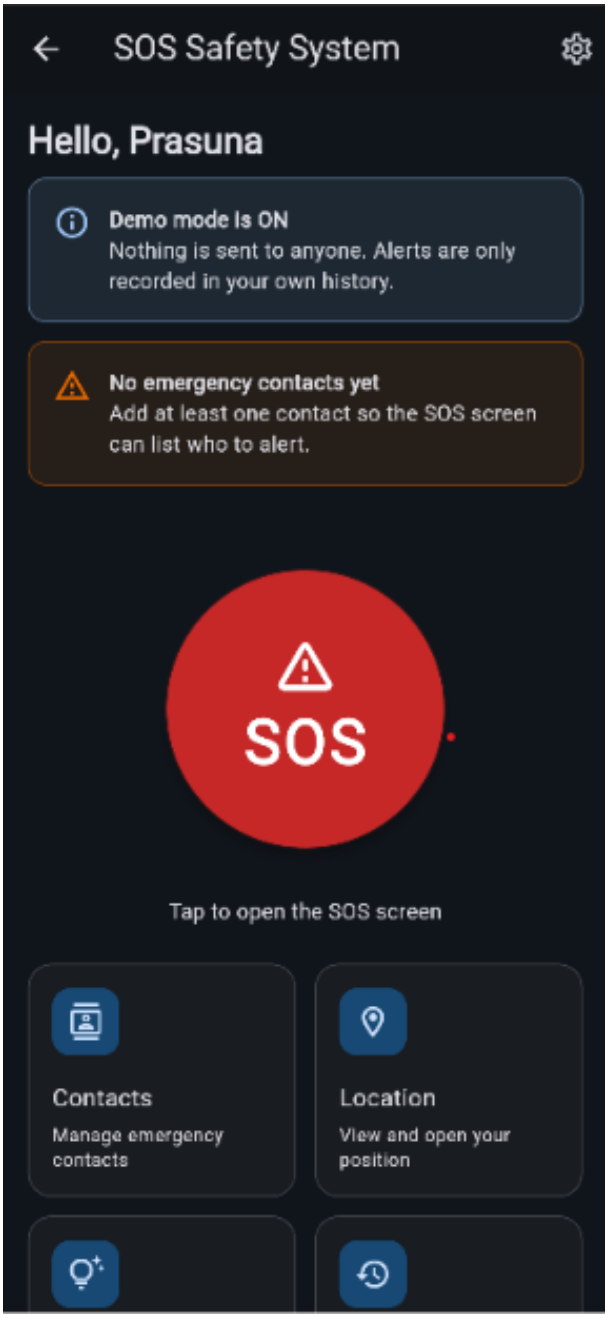
```
      ),
    );
  }
}

class SafetyDashboard extends StatelessWidget {
  const SafetyDashboard({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('SOS Safety')),
      body: GridView.count(
        crossAxisCount: 2,
        children: const [
          SafetyCard(
            title: 'Emergency SOS',
            icon: Icons.sos,
            color: Colors.red,
          ),
          SafetyCard(
            title: 'Contacts',
            icon: Icons.contacts,
            color: Colors.blue,
          ),
          SafetyCard(
            title: 'Location',
            icon: Icons.location_on,
            color: Colors.green,
          ),
          SafetyCard(
            title: 'Safety Tips',
            icon: Icons.security,
            color: Colors.orange,
          ),
        ],
      ),
    );
  }
}
```

Output:

Reusable feature tiles can be displayed for Contacts, Location, Safety Tips, History and Settings.



Result:

Thus, reusable custom widgets were created to organize the SOS application interface.

EXPERIMENT 6(b)

THEMES AND CUSTOM STYLES

Aim:

To apply consistent themes and custom styles to the application.

Description:

ThemeData defines the common colors, typography and component appearance of a Flutter application. The SOS Safety System defines light and dark themes in its AppTheme class and allows the theme mode to be controlled through settings.

Program:

```dart
import 'package:flutter/material.dart';

void main() {
  runApp(const SafetyApp());
}

class SafetyApp extends StatelessWidget {
  const SafetyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      theme: ThemeData(
        useMaterial3: true,
        colorScheme: ColorScheme.fromSeed(
          seedColor: Colors.red,
        ),
        appBarTheme: const AppBarTheme(
          backgroundColor: Colors.red,
          foregroundColor: Colors.white,
        ),
        elevatedButtonTheme: ElevatedButtonThemeData(
          style: ElevatedButton.styleFrom(
            backgroundColor: Colors.red,
            foregroundColor: Colors.white,
          ),
        ),
      ),
      home: const SafetyHome(),
    );
  }
```

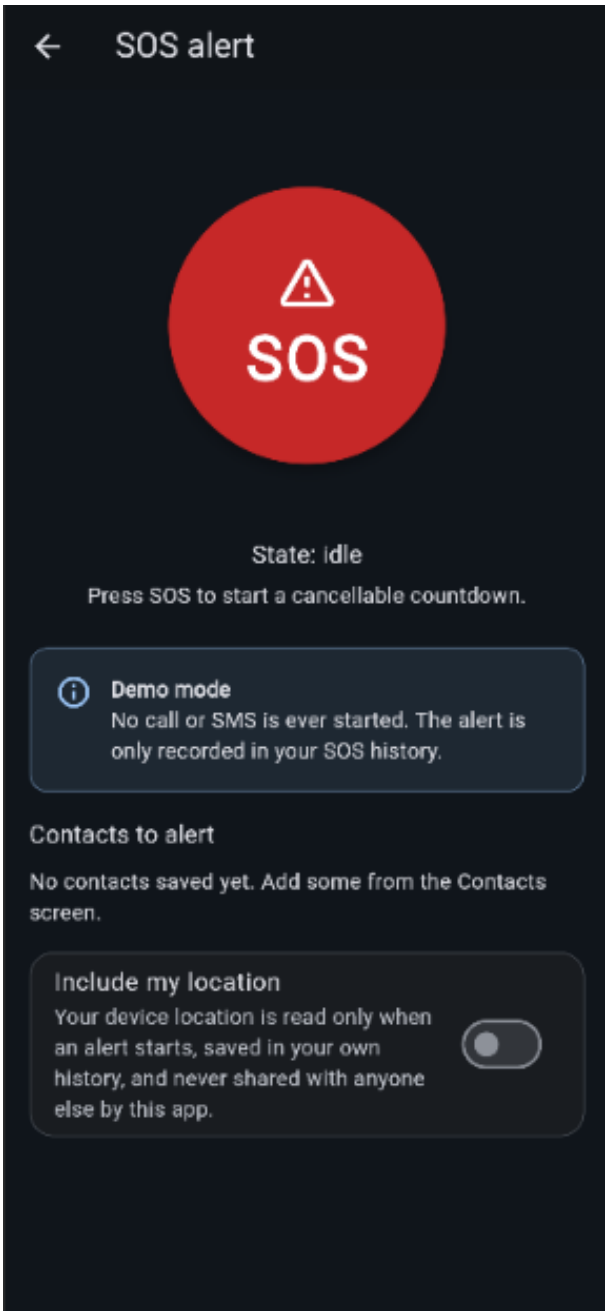
```dart
}

class SafetyHome extends StatelessWidget {
  const SafetyHome({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('SOS Safety System'),
      ),
      body: Center(
        child: Column(
          mainAxisAlignment: MainAxisAlignment.center,
          children: [
            const Icon(
              Icons.health_and_safety,
              size: 80,
              color: Colors.red,
            ),
            const SizedBox(height: 20),
            const Text(
              'Stay Safe, Stay Alert',
              style: TextStyle(
                fontSize: 23,
                fontWeight: FontWeight.bold,
              ),
            ),
            const SizedBox(height: 20),
            ElevatedButton(
              onPressed: () {},
              child: const Text('Emergency SOS'),
            ),
          ],
        ),
      ),
    );
  }
}
```

Output:

The application displays its interface using the selected light or dark theme.



Result:

Thus, centralized theme configuration was implemented using ThemeData.

EXPERIMENT 7(a)

CREATING FORMS WITH INPUT FIELDS

Aim:

To create input forms for entering emergency contact details.

Description:

Forms collect information from users. The SOS Safety System provides a contact form to enter and manage emergency contact information. The form screen uses Flutter input widgets and controllers.

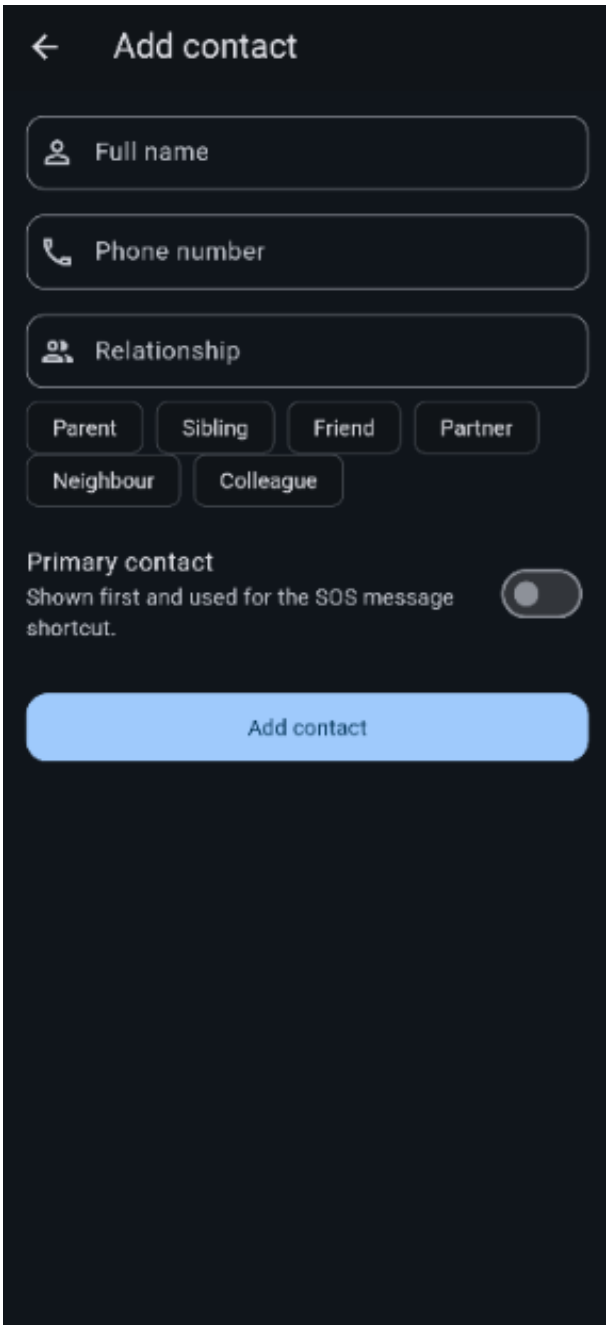
Program:

```
final nameController = TextEditingController();
final phoneController = TextEditingController();

Column(
  children: [
    TextFormField(
      controller: nameController,
      decoration: const InputDecoration(
        labelText: 'Contact Name',
      ),
    ),
    TextFormField(
      controller: phoneController,
      keyboardType: TextInputType.phone,
      decoration: const InputDecoration(
        labelText: 'Phone Number',
      ),
    ),
    ElevatedButton(
      onPressed: () {
        // Submit the entered contact details.
      },
      child: const Text('Save Contact'),
    ),
  ],
)
```

Output:

The user can enter a contact name and phone number in the emergency contact form.



Result:

Thus, input fields were used to collect emergency contact details.

EXPERIMENT 7(b)

FORM VALIDATION AND ERROR HANDLING

Aim:

To validate user input and handle invalid form data.

Description:

Form validation checks whether the information entered by a user satisfies the required conditions. The project contains reusable validation functions in its validators utility.

Program:

```dart
import 'package:flutter/material.dart';

class ContactForm extends StatefulWidget {
  const ContactForm({super.key});

  @override
  State<ContactForm> createState() => _ContactFormState();
}

class _ContactFormState extends State<ContactForm> {
  final key = GlobalKey<FormState>();
  final name = TextEditingController();
  final phone = TextEditingController();

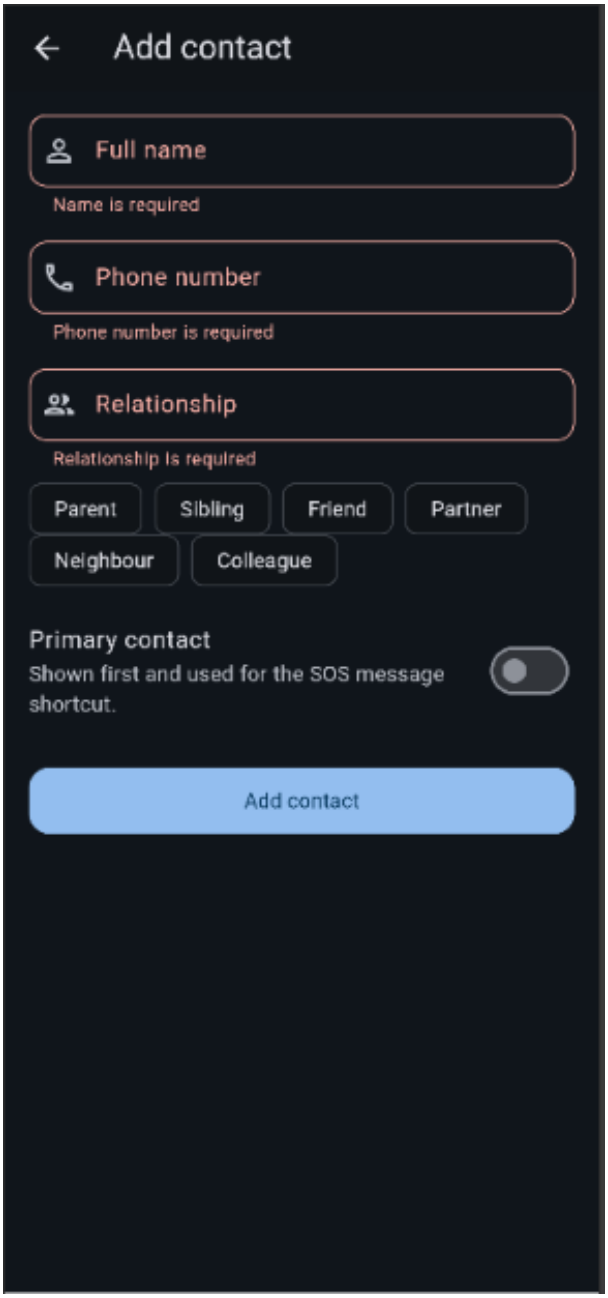
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Emergency Contact')),
      body: Padding(
        padding: const EdgeInsets.all(16),
        child: Form(
          key: key,
          child: Column(children: [
            TextFormField(
              controller: name,
              decoration: const InputDecoration(labelText: 'Name'),
              validator: (v) =>
                  v!.isEmpty ? 'Enter name' : null,
            ),
            TextFormField(
              controller: phone,
              keyboardType: TextInputType.phone,
              decoration: const InputDecoration(labelText: 'Phone'),
```

```
              validator: (v) =>
                  RegExp(r'^[0-9]{10}$').hasMatch(v!)
                      ? null : 'Enter valid 10-digit number',
            ),
            ElevatedButton(
              onPressed: () {
                if (key.currentState!.validate()) {
                  ScaffoldMessenger.of(context).showSnackBar(
                    const SnackBar(content: Text('Valid Contact')),
                  );
                }
              },
              child: const Text('Validate'),
            ),
          ]),
        ),
      ),
    );
  }

  @override
  void dispose() {
    name.dispose();
    phone.dispose();
    super.dispose();
  }
}
```

Output:

The form displays validation messages when required information is missing or the phone number is invalid.



Result:

Thus, input validation was implemented using reusable validation functions.

EXPERIMENT 8(a)

IMPLEMENTING UI ANIMATIONS

Aim:

To implement animations in the Flutter application.

Description:

Animations improve the user experience by creating smooth visual transitions. The SOS Safety System contains a reusable FadeSlideIn widget to animate the appearance of interface content.

Program:

```dart
import 'package:flutter/material.dart';

class AnimationDemo extends StatefulWidget {
  const AnimationDemo({super.key});

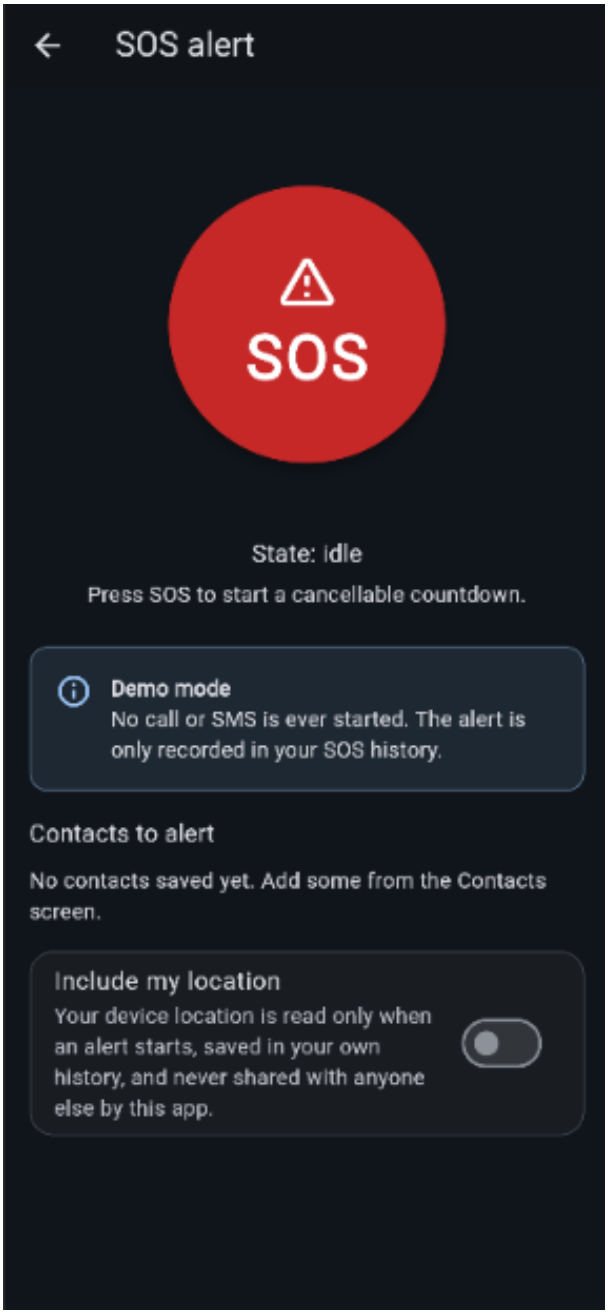
  @override
  State<AnimationDemo> createState() => _AnimationDemoState();
}

class _AnimationDemoState extends State<AnimationDemo> {
  bool visible = true;

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      body: Center(
        child: AnimatedOpacity(
          opacity: visible ? 1 : 0,
          duration: const Duration(milliseconds: 500),
          child: const Icon(
            Icons.health_and_safety,
            size: 80,
          ),
        ),
      ),
      floatingActionButton: FloatingActionButton(
        onPressed: () => setState(() => visible = !visible),
        child: const Icon(Icons.play_arrow),
      ),
    );
  }
}
```

Output:

The safety icon smoothly changes its opacity when the button is pressed.



Result:

Thus, an implicit animation was implemented using AnimatedOpacity.

EXPERIMENT 8(b)

PAGE TRANSITION ANIMATIONS

Aim:

To implement animated transitions while navigating between application screens.

Description:

Flutter provides animation classes to create transitions between pages. The AppRouter in the SOS Safety System uses PageRouteBuilder with FadeTransition and SlideTransition.

Program:

```dart
import 'package:flutter/material.dart';

class SafetyAnimation extends StatefulWidget {
  const SafetyAnimation({super.key});

  @override
  State<SafetyAnimation> createState() =>
      _SafetyAnimationState();
}

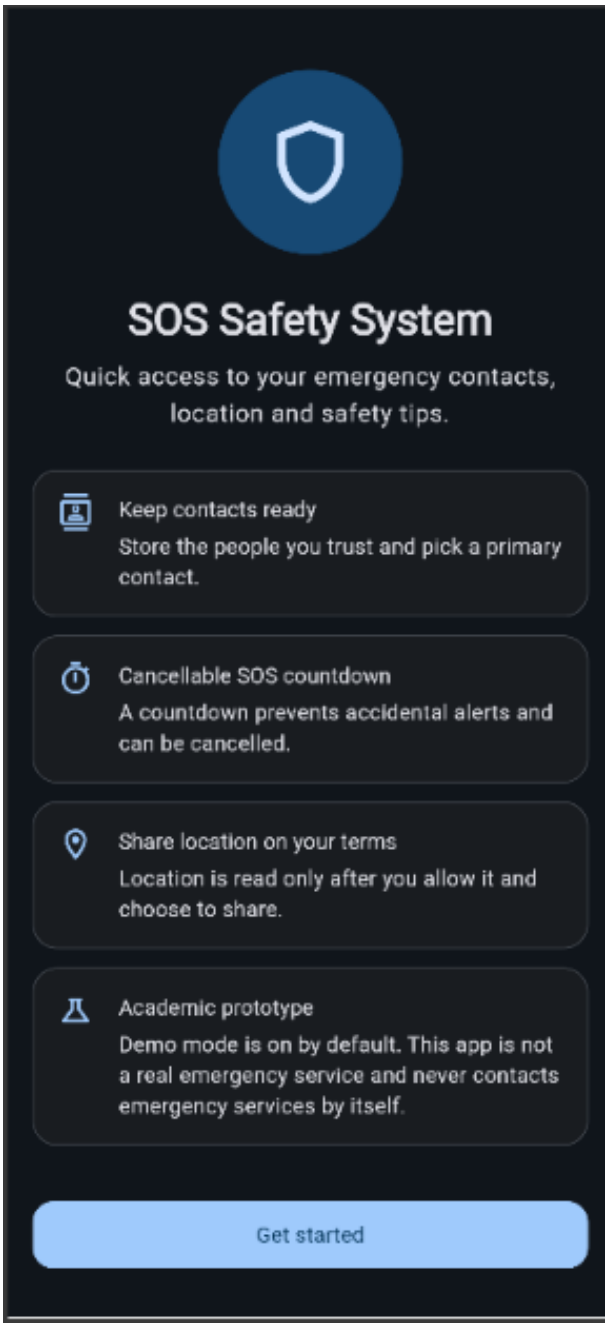
class _SafetyAnimationState extends State<SafetyAnimation> {
  bool showCard = false;

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Safety Animation')),
      body: Center(
        child: Column(
          mainAxisAlignment: MainAxisAlignment.center,
          children: [
            AnimatedSlide(
              offset: showCard
                  ? Offset.zero
                  : const Offset(0, 1),
              duration: const Duration(milliseconds: 500),
              child: const Card(
                child: Padding(
                  padding: EdgeInsets.all(24),
                  child: Column(
                    children: [
                      Icon(
                        Icons.health_and_safety,
```

```
                    size: 60,
                    color: Colors.red,
                  ),
                  Text('Stay Safe'),
                ],
              ),
            ),
          ),
        ),
        const SizedBox(height: 20),
        ElevatedButton(
          onPressed: () {
            setState(() {
              showCard = !showCard;
            });
          },
          child: const Text('Toggle Animation'),
        ),
      ],
    ),
  ),
 );
 }
}
```

Output:

Application screens appear with a fade and slide transition during navigation.



Result:

Thus, page transition animations were implemented using Flutter animation widgets.

EXPERIMENT 9(a)

CONNECTING THE APPLICATION TO FIREBASE

Aim:

To initialize Firebase services in the SOS Safety System.

Description:

Firebase provides backend services for mobile applications. The project uses Firebase Core, Firebase Authentication, Cloud Firestore and Firebase Messaging. Firebase is initialized before the application starts.

Program:

```
    import 'dart:convert';
import 'package:flutter/material.dart';
import 'package:http/http.dart' as http;

class ApiService {
  static Future<List<dynamic>> fetchPosts() async {
    final response = await http.get(
      Uri.parse('https://jsonplaceholder.typicode.com/posts'),
    );

    if (response.statusCode == 200) {
      return jsonDecode(response.body) as List<dynamic>;
    } else {
      throw Exception('Failed to load data');
    }
  }
}

class ApiDemo extends StatefulWidget {
  const ApiDemo({super.key});

  @override
  State<ApiDemo> createState() => _ApiDemoState();
}

class _ApiDemoState extends State<ApiDemo> {
  String result = 'Press the button to fetch data';

  Future<void> getData() async {
    try {
      final posts = await ApiService.fetchPosts();
      setState(() {
```

```
        result = posts.take(3)
            .map((post) => post['title'])
            .join('\n\n');
      });
    } catch (e) {
      setState(() {
        result = 'Unable to fetch data';
      });
    }
  }

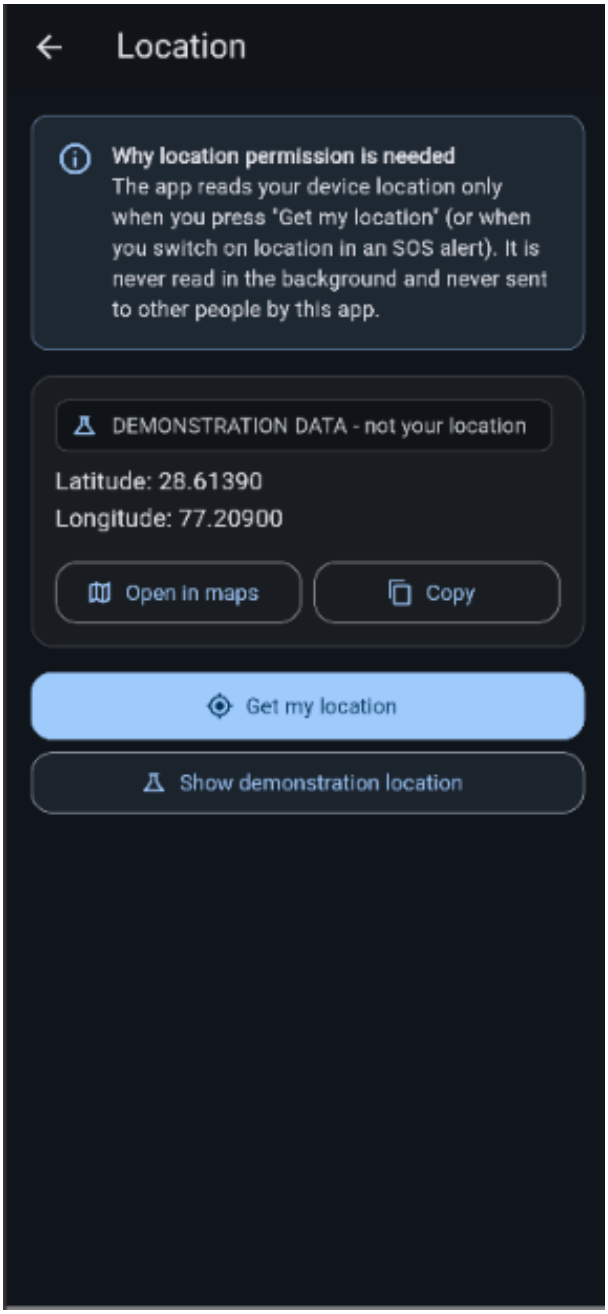
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('API Connection')),
      body: Center(
        child: Padding(
          padding: const EdgeInsets.all(16),
          child: Column(
            mainAxisAlignment: MainAxisAlignment.center,
            children: [
              Text(result),
              const SizedBox(height: 20),
              ElevatedButton(
                onPressed: getData,
                child: const Text('Fetch API Data'),
              ),
            ],
          ),
        ),
      ),
    );
  }
}
```

Execution Command:

`flutter pub add http`

Output:

Firebase is initialized before the SOS application is launched, provided the platform configuration is valid.



Result:

Thus, Firebase initialization was implemented in the Flutter application.

EXPERIMENT 9(b)

RETRIEVING AND DISPLAYING APPLICATION DATA

Aim:

To retrieve application data from Firebase and display it in the user interface.

Description:

Cloud Firestore is used by the project repository to store and retrieve application data. The providers expose data to the interface and notify widgets when the data changes.

Program:

```dart
import 'package:flutter/material.dart';

class ApiDisplay extends StatelessWidget {
  const ApiDisplay({super.key});

  Future<List<dynamic>> fetchPosts() async {
    final posts = await ApiService.fetchPosts();
    return posts.take(5).toList();
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Safety Information')),
      body: FutureBuilder<List<dynamic>>(
        future: fetchPosts(),
        builder: (context, snapshot) {
          if (snapshot.connectionState ==
              ConnectionState.waiting) {
            return const Center(
              child: CircularProgressIndicator(),
            );
          }

          if (snapshot.hasError) {
            return const Center(
              child: Text('Unable to load data'),
            );
          }

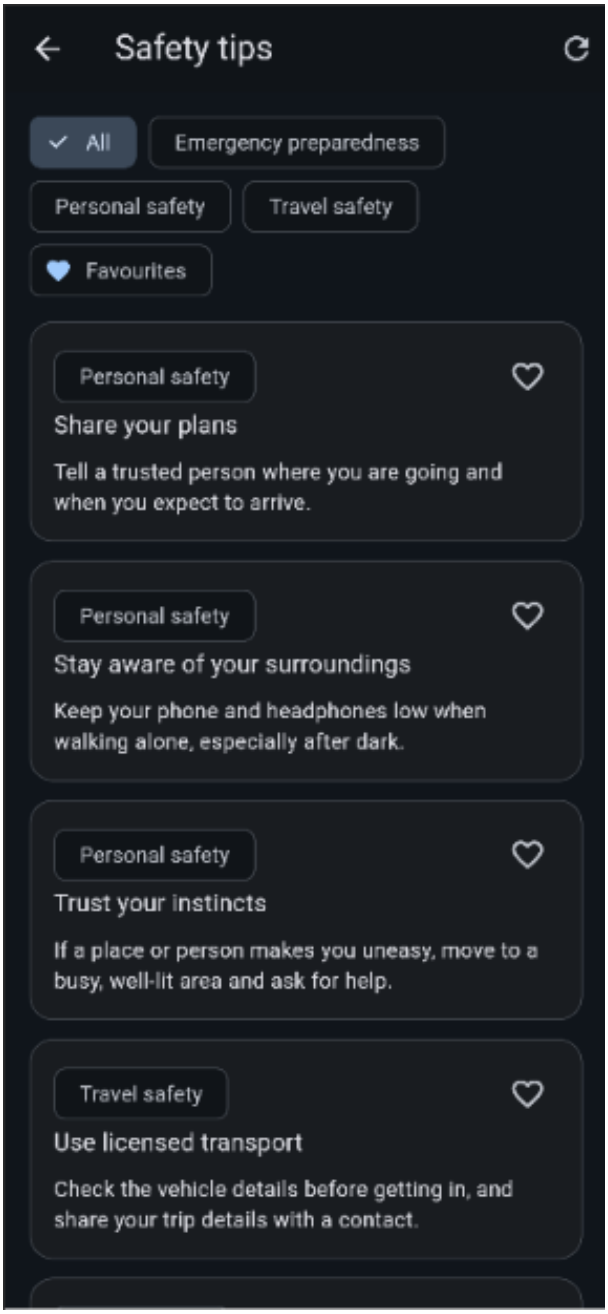
          final posts = snapshot.data ?? [];

          return ListView.builder(
```

```
            itemCount: posts.length,
            itemBuilder: (context, index) {
              return Card(
                child: ListTile(
                  leading: const Icon(
                    Icons.security,
                    color: Colors.red,
                  ),
                  title: Text(posts[index]['title']),
                  subtitle: Text(posts[index]['body']),
                ),
              );
            },
          );
        },
      ),
    );
  }
}
```

Output:

The application can retrieve records from the configured Firestore collection and display them through its data providers and screens.



Result:

Thus, Firestore data access was demonstrated in the SOS Safety System.

EXPERIMENT 10(a)

UNIT TESTING IN FLUTTER

Aim:

To understand unit testing and verify individual functions in a Flutter application.

Description:

Unit testing checks whether individual functions produce expected results. Flutter provides the flutter_test package for writing and executing tests. Validation utilities and application logic can be tested independently.

Program:

```dart
import 'package:flutter_test/flutter_test.dart';

String? validateRequired(String? value) {
  if (value == null || value.trim().isEmpty) {
    return 'This field is required';
  }
  return null;
}

void main() {
  test('Empty value should return an error', () {
    expect(validateRequired(''), isNotNull);
  });

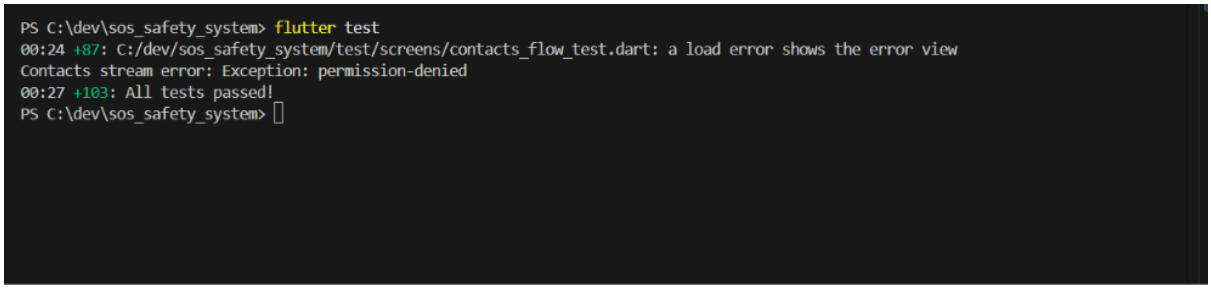
  test('Non-empty value should be accepted', () {
    expect(validateRequired('Emergency Contact'), isNull);
  });
}
```

Execution Command:

`flutter test`

Output:

The tests pass when the validation function returns the expected values. Run the tests in the project to record the actual result.



Result:

Thus, unit testing of a validation function was demonstrated using flutter_test.

EXPERIMENT 10(b)

DEBUGGING A FLUTTER APPLICATION

Aim:

To identify and correct errors in a Flutter application using debugging techniques.

Description:

Debugging is the process of identifying and fixing errors. Flutter developers can use debugPrint, breakpoints, the Debug Console and Flutter DevTools. The SOS project also handles Firebase initialization errors by displaying an error screen instead of a blank screen.

Program:

```
try {
  await Firebase.initializeApp(
    options: DefaultFirebaseOptions.currentPlatform,
  );
} catch (error) {
  debugPrint('Firebase initialisation failed: $error');

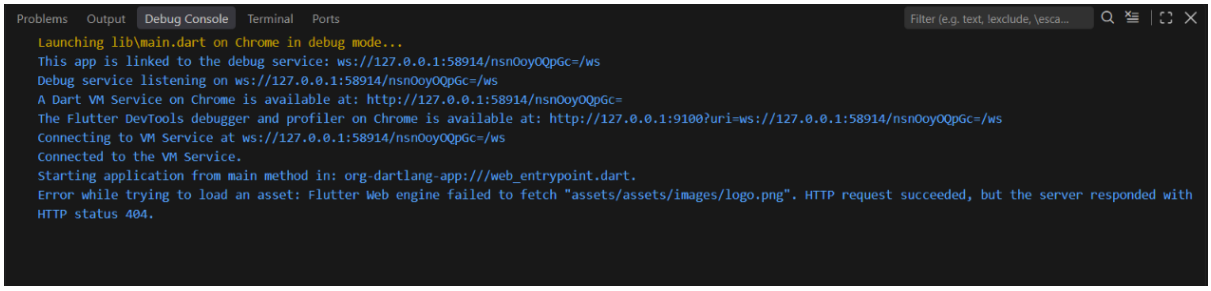
  runApp(_FirebaseErrorApp(error: error));
  return;
}
```

Debugging Procedure:

1. Open the SOS Safety System project in VS Code or Android Studio.
2. Run the application in debug mode.
3. Check the Debug Console for error messages.
4. Set breakpoints to inspect variable values.
5. Correct the errors and run the application again.

Output:

If Firebase initialization fails, the application displays a dedicated error screen and prints the error in the Debug Console.



Result:

Thus, error handling and debugging techniques were demonstrated.

EXPERIMENT 11

CASE STUDY: SOS SAFETY SYSTEM

1. Introduction

SOS Safety System is a Flutter-based mobile application designed to provide users with convenient access to emergency-related features. It includes user authentication, emergency contact management, SOS countdown, location access, safety tips, SOS history and application settings.

2. Problem Statement

During an emergency, users may need quick access to trusted contacts, their current location and emergency-related actions. Searching for these features separately can take time. The SOS Safety System brings these functions together in a single mobile application.

3. Objectives

- To provide user registration and login.
- To allow users to manage emergency contacts.
- To provide an SOS screen with a cancellable countdown.
- To access the device location with permission.
- To maintain SOS history.
- To provide safety tips and application settings.

4. Proposed System

The proposed system is developed using Flutter and Dart. Firebase Authentication is used for user authentication, Cloud Firestore is used as the cloud data repository, and Geolocator is used for location services. Provider is used for state management. The SOS screen supports demo and live modes. In live mode, the application opens the phone dialer or messaging application for the user to complete the call or message manually. Opening an external application does not confirm that a call or message was completed or delivered.

5. Technologies Used

- **Flutter:** Mobile application user interface.
- **Dart:** Application programming language.
- **Firebase Authentication:** User authentication.
- **Cloud Firestore:** Cloud data storage.
- **Firebase Messaging:** Messaging and notification support.
- **Geolocator:** Device location access.
- **Provider:** Application state management.
- **Shared Preferences:** Local preference storage.
- **HTTP:** Communication with the safety tips API.

6. Main Modules

1. **Authentication Module:** Supports registration, login and password reset.
2. **Home Dashboard:** Displays the main safety features and application status.
3. **SOS Module:** Provides a cancellable countdown and alert actions.
4. **Emergency Contacts:** Allows users to add, edit and manage trusted contacts.
5. **Location Module:** Retrieves the device location after permission is granted.
6. **Safety Tips:** Displays safety-related information.
7. **SOS History:** Displays recorded SOS events.
8. **Settings:** Manages profile, theme and privacy-related preferences.

7. System Architecture

The SOS Safety System follows a layered architecture. The Flutter presentation layer provides the user interface for login, registration, dashboard, SOS, emergency contacts, location, safety tips, history and settings.

Provider is used for state management, while application services handle authentication, contact management, location access and SOS-related operations. Firebase services provide authentication, cloud data storage and messaging support.

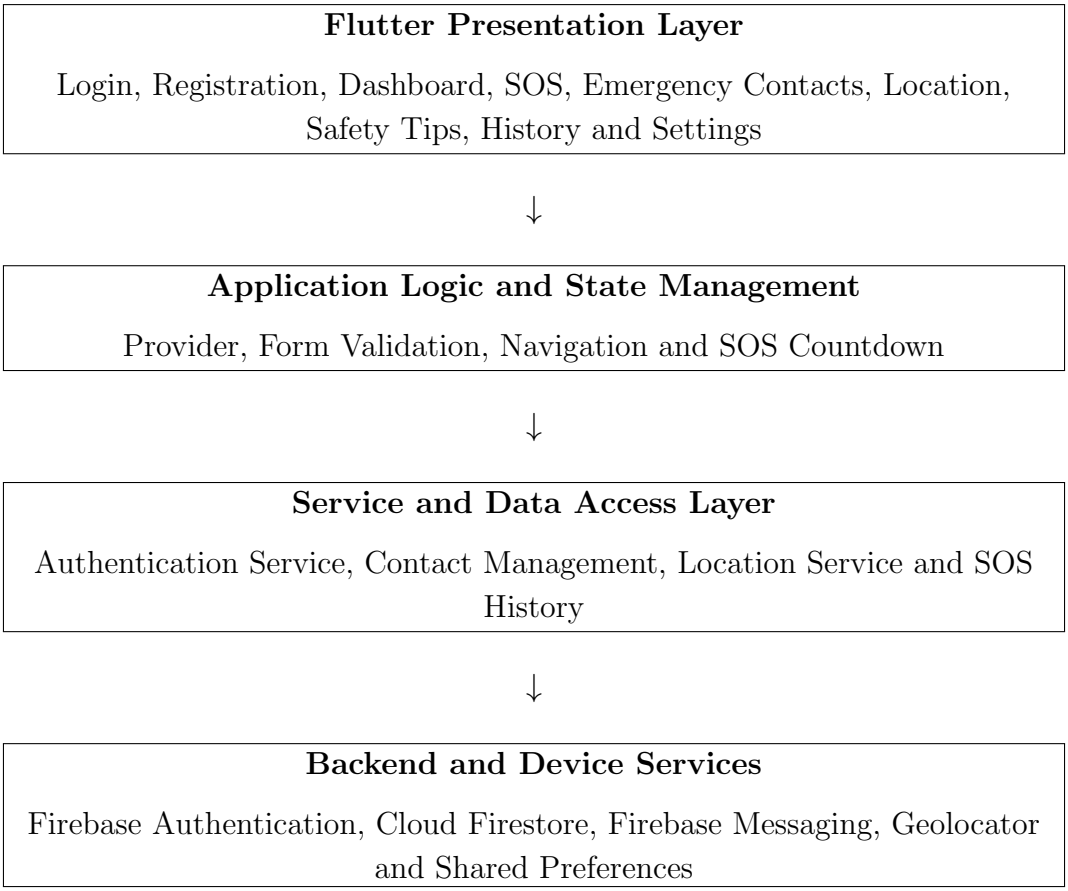


Figure 1: SOS Safety System High-Level Architecture

8. Database Design

The SOS Safety System uses Cloud Firestore as its cloud data repository. The logical database design consists of user profiles, emergency contacts, SOS events, safety tips and user settings. These collections help organize and manage application information.

Logical Database Collections

- **Users:** Stores user account details and profile information.
- **Emergency Contacts:** Stores contact names, phone numbers and relationship details associated with a user.
- **SOS Events:** Stores SOS event information, timestamps and available event status details.
- **Safety Tips:** Stores safety-related information displayed in the application.
- **User Settings:** Stores application preferences such as theme and privacy settings.

Database Relationships

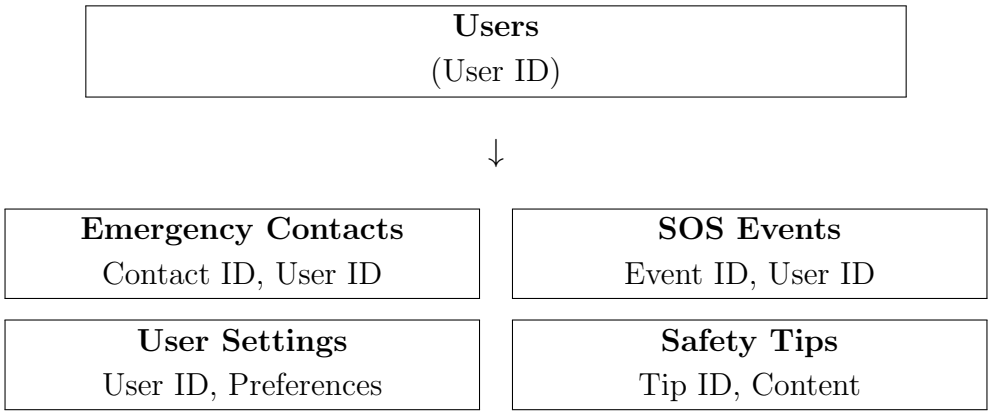


Figure 2: Logical SOS Safety System Database Relationships

9. User Workflow

The user workflow describes the sequence of operations performed by a user while using the SOS Safety System. The application allows users to register, log in, manage emergency contacts and access emergency-related features.

Working Procedure

1. The user launches the SOS Safety System application.
2. The user registers a new account or logs in using existing credentials.
3. After successful authentication, the home dashboard is displayed.
4. The user can add, edit and manage emergency contacts.
5. The user opens the SOS screen and starts the emergency countdown.
6. The countdown can be cancelled before the alert action is initiated.
7. In live mode, the application provides options to open the phone dialer or messaging application.
8. The user completes the call or message manually through the external application.
9. The user can also access location information, safety tips, SOS history and application settings.

User Workflow Diagram

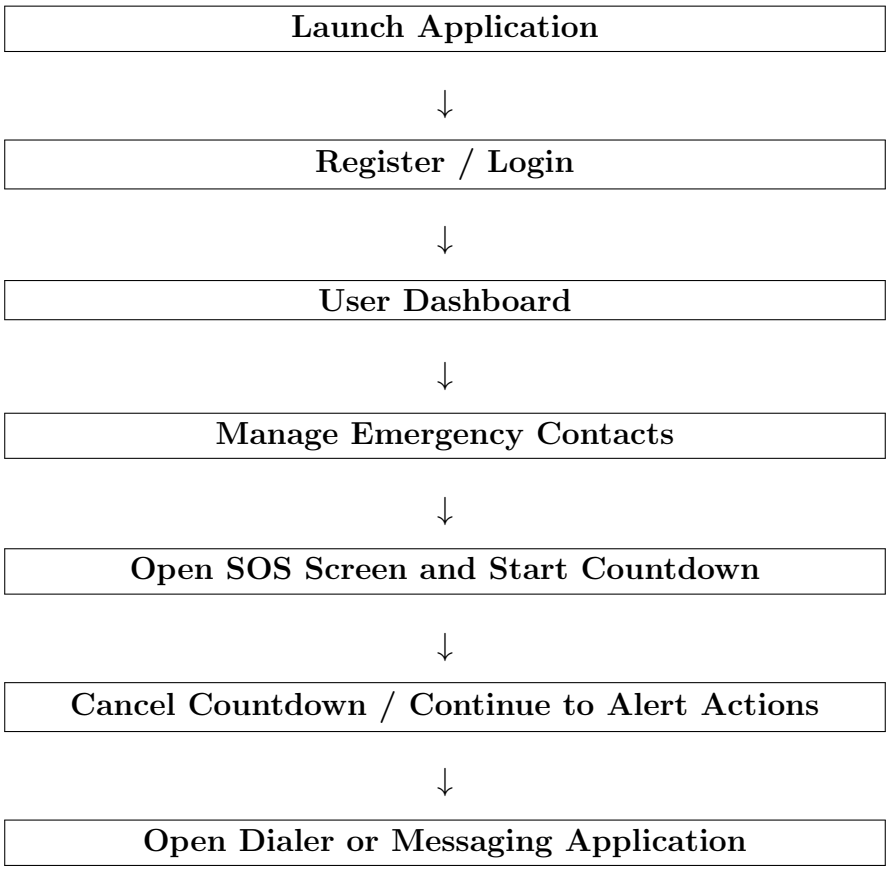


Figure 3: User Workflow of SOS Safety System

10. Android and Chrome Comparison

The SOS Safety System was developed using Flutter, which supports multiple platforms through a single codebase. The application was tested on an Android mobile device and Google Chrome desktop browser. The screenshots below illustrate the user interface on both platforms.

Welcome Screen:

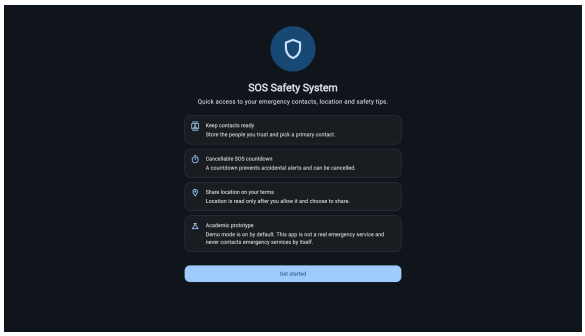


Figure 7: Desktop View

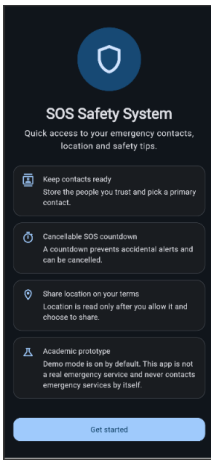


Figure 8: Mobile View

Login Screen:

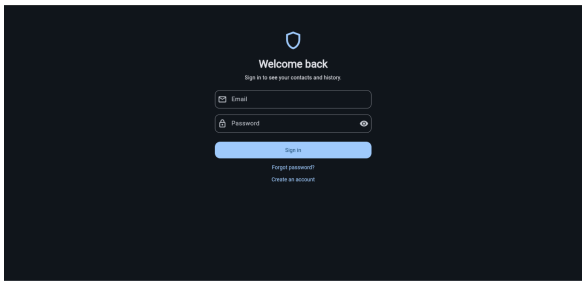


Figure 9: Desktop View

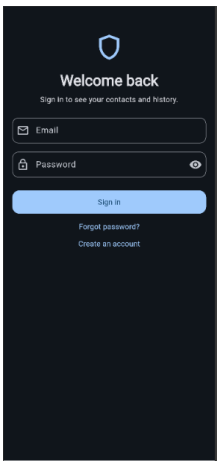


Figure 10: Mobile View

Registration Screen:

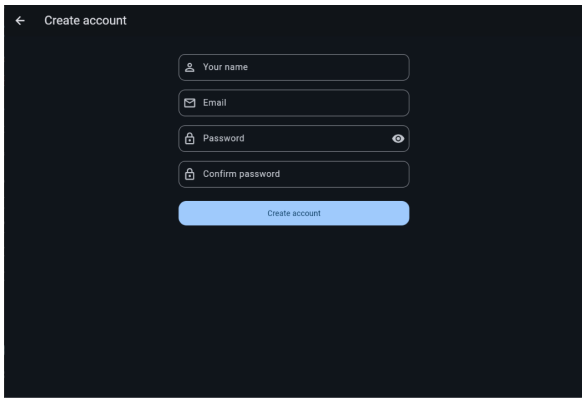


Figure 11: Desktop View

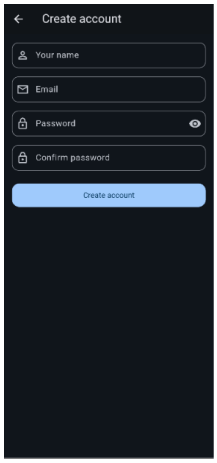


Figure 12: Mobile View

Dashboard Screen:

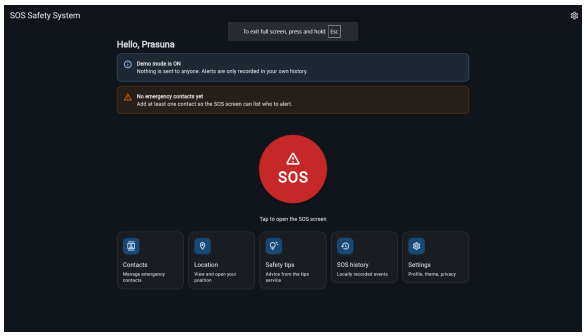


Figure 13: Desktop View

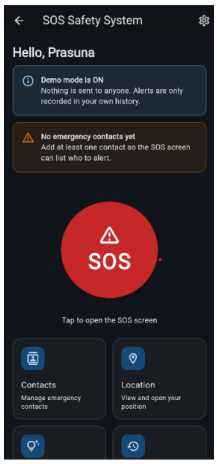


Figure 14: Mobile View

SOS Screen:

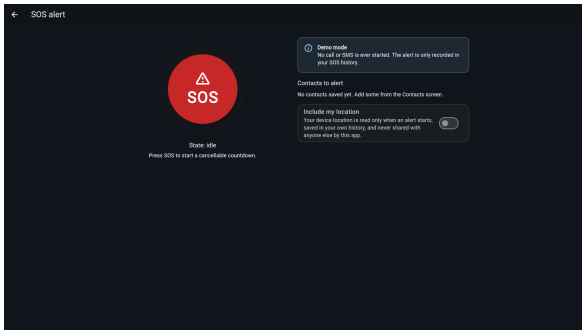


Figure 15: Desktop View

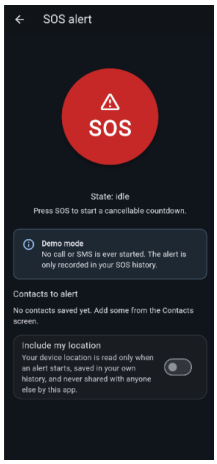


Figure 16: Mobile View

Emergency Contacts Screen:

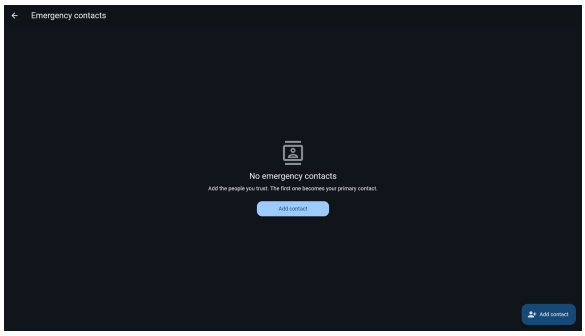


Figure 17: Desktop View

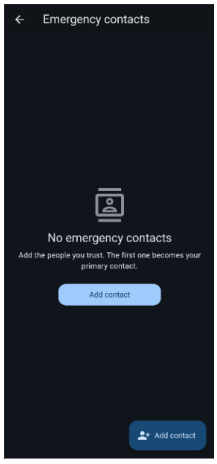


Figure 18: Mobile View

Add Emergency Contact Screen:

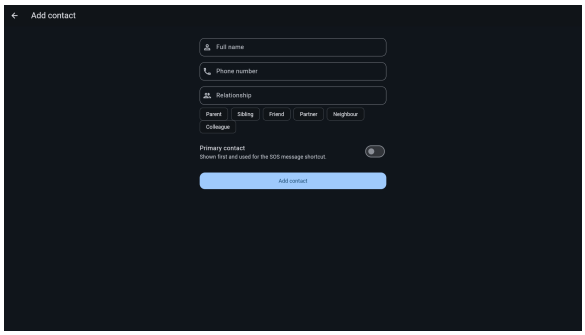


Figure 19: Desktop View

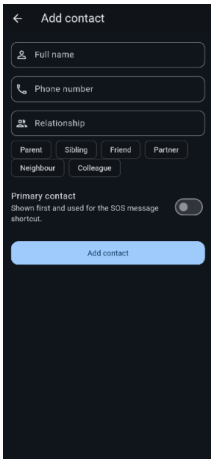


Figure 20: Mobile View

Location Screen:

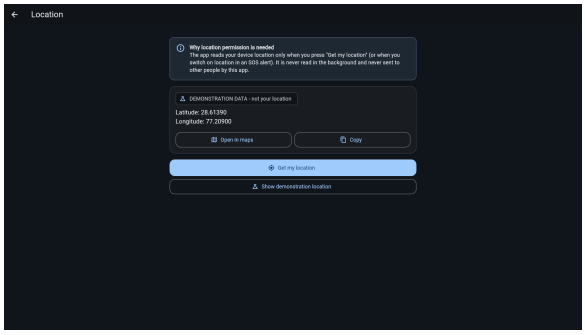


Figure 21: Desktop View

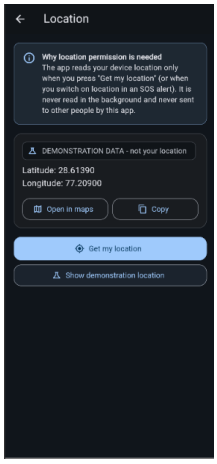


Figure 22: Mobile View

Safety Tips Screen:

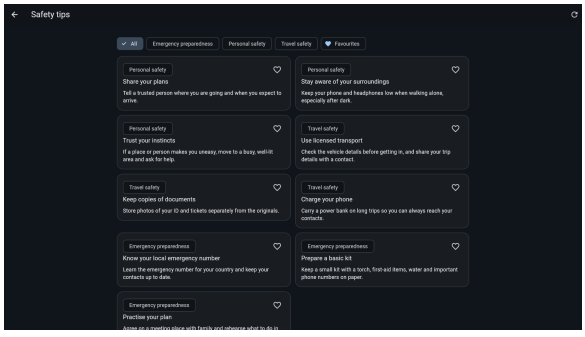


Figure 23: Desktop View

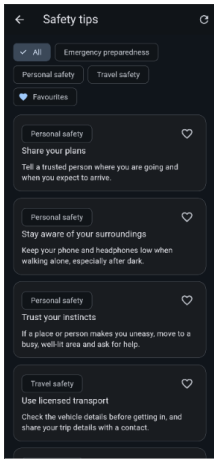


Figure 24: Mobile View

Settings Screen:

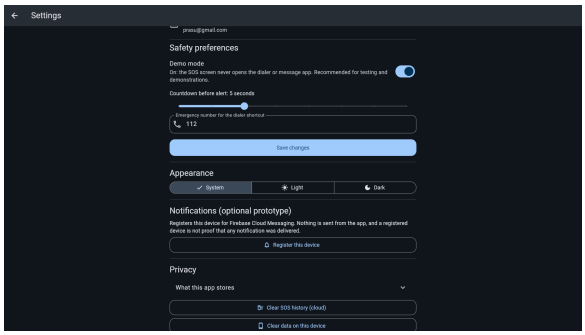


Figure 25: Desktop View

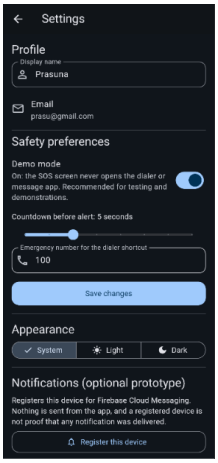


Figure 26: Mobile View

11. Security and Privacy Considerations

The SOS Safety System handles sensitive user information such as login credentials, emergency contact details, and location data. Therefore, security and privacy are important aspects of the application. The system uses Firebase services and device permissions to support secure access and controlled use of personal information.

1. User Authentication:

Firebase Authentication is used to support user registration and login. Users must authenticate before accessing their personal application features.

2. Database Security:

Cloud Firestore is used to store application data. Firebase Security Rules should be configured to ensure that authenticated users can access only their authorized data. Unauthorized database access must be restricted.

3. Location Privacy:

The application uses the Geolocator package to access device location. Location permission must be requested from the user before accessing location information. Users should be able to deny location access when they do not wish to share it.

4. Emergency Contact Protection:

Emergency contact details are personal information and should be handled carefully. Access to contact information should be limited to authorized users. The application should not expose contact details to other users.

5. Secure Data Handling:

User information should be transmitted and stored using appropriate security mecha-

nisms. Passwords must not be stored as plain text. Sensitive information should not be unnecessarily displayed or shared.

6. External Calling and Messaging:

During an SOS action, the application can open the device dialer or messaging application. The user must manually complete the call or message. The application should not assume that an emergency call or message has been successfully delivered.

7. User Privacy:

The application should collect only the information required for its functionality. Users should be informed about location and contact permissions and how their information is used.

12. Testing Plan

Testing is an important phase in the development of the SOS Safety System. It helps verify that the application functions correctly and that its major features work as expected on Android and Google Chrome.

The following test cases are planned to evaluate the functionality and reliability of the application.

Test Cases:

Test ID	**Test Scenario**	**Expected Result**
TC01	User Registration	The application should allow a new user to register with valid details.
TC02	User Login	The application should authenticate valid users and allow them to access the dashboard.
TC03	Invalid Login	The application should display an appropriate error message for invalid login details.
TC04	Emergency Contact Management	The user should be able to add and view emergency contact details.
TC05	SOS Countdown	The SOS screen should start the countdown when the SOS action is initiated.

TC06	Cancel SOS Countdown	The user should be able to cancel the countdown before proceeding to the alert action.
TC07	Location Permission	The application should request location permission before accessing the device location.
TC08	Location Permission Denied	The application should handle denied location permission without crashing.
TC09	Safety Tips	The application should display available safety tips to the user.
TC10	Settings Screen	The application should display the available settings and apply supported preferences.
TC11	SOS History	The application should display available SOS history records to the user.
TC12	Android Compatibility	The application should display its interface correctly on an Android device.
TC13	Chrome Compatibility	The application should display its interface correctly in the Chrome browser.

Testing Methods:

- **Functional Testing:** Verifies individual features such as registration, login, emergency contact management, and SOS countdown.
- **UI Testing:** Checks the appearance and usability of application screens on mobile and desktop layouts.
- **Permission Testing:** Verifies application behaviour when location permission is granted or denied.
- **Platform Testing:** Checks application behaviour on Android devices and Google Chrome.
- **Error Handling Testing:** Checks how the application responds to invalid inputs, missing permissions, and network connectivity problems.

The testing process helps identify functional errors, interface issues, and platform-related limitations. The test cases should be executed on the target platforms and their actual results should be recorded.

13. Advantages

The SOS Safety System provides several advantages by combining emergency-related features into a single mobile and web application. It helps users access important safety features through a simple and user-friendly interface.

1. Quick Access to SOS Feature:

The application provides a dedicated SOS feature that allows users to initiate an emergency action through a countdown mechanism.

2. Emergency Contact Management:

Users can add and manage emergency contact details in one place, making important contact information easier to access.

3. Location Access:

The application uses the Geolocator package to obtain device location after receiving the required permission from the user.

4. User-Friendly Interface:

The application provides a simple and organized interface that helps users navigate between different safety features.

5. Cross-Platform Support:

Flutter allows the application to run on Android and Google Chrome using a common codebase, reducing the need to develop separate applications for each platform.

6. Safety Awareness:

The Safety Tips module provides useful safety-related information that helps users become more aware of personal safety practices.

7. Cloud-Based Data Management:

Firebase services support user authentication and cloud-based data storage, allowing application data to be managed through a centralized backend.

8. SOS History:

The application provides access to available SOS history records, helping users review their previous emergency-related activities.

9. Customizable Settings:

The Settings module provides users with access to available application preferences and

configuration options.

10. Expandability:

The application is developed using Flutter and Firebase, making it possible to extend its functionality with additional features in future versions.

14. Limitations

Although the SOS Safety System provides useful safety-related features, it has certain limitations that may affect its functionality and reliability.

1. Internet Connectivity:

Some application features depend on internet connectivity for communication with Firebase services. Network interruptions may affect data access and synchronization.

2. Location Accuracy:

The accuracy of location information depends on the device's GPS capabilities, location settings, and environmental conditions. Incorrect or unavailable location data may affect location-based features.

3. Location Permission:

The application requires user permission to access device location. If permission is denied, location-related features may not work as expected.

4. Manual Emergency Actions:

The application opens the device dialer or messaging application during an SOS action. The user must manually complete the call or message. Therefore, the application cannot guarantee that an emergency call or message has been completed or delivered.

5. Platform Differences:

The behaviour of certain features may differ between Android devices and Google Chrome because of differences in device permissions, browser capabilities, and operating system support.

6. Device Compatibility:

Application performance and interface behaviour may vary depending on device specifications, screen dimensions, operating system versions, and browser versions.

7. Dependency on Firebase Services:

Authentication and cloud data operations depend on the availability and correct configuration of Firebase services. Service interruptions or configuration errors may affect application functionality.

8. Limited Emergency Response:

The application provides tools to initiate emergency-related actions, but it does not guarantee that emergency services or emergency contacts will receive or respond to an alert.

15. Future Enhancements

The SOS Safety System can be improved in future versions by adding advanced features to enhance its functionality, reliability, accessibility, and user experience.

1. Automatic Emergency Notifications:

A future version can include an automated notification mechanism to send emergency alerts to registered contacts after the user confirms an SOS action.

2. Live Location Sharing:

Real-time location sharing can be introduced to help emergency contacts track the user's location during an emergency situation.

3. Admin Dashboard:

An administrator dashboard can be developed to manage application content, monitor system activities, and maintain safety-related information.

4. Offline Support:

Offline capabilities can be added to provide access to selected safety information and emergency contact details when internet connectivity is unavailable.

5. Multilingual Support:

The application can support multiple languages to make its features accessible to users from different regions.

6. Voice-Activated SOS:

A voice-activated emergency feature can be explored to allow users to initiate an SOS action through voice commands, subject to device permissions and platform support.

7. Improved Security:

Additional security measures, such as stronger access controls, secure session management, and regular security testing, can be introduced to improve data protection.

8. Enhanced Testing:

More automated unit, widget, and integration tests can be added to improve application reliability across different devices and platforms.

9. Wearable Device Integration:

Future versions may support compatible wearable devices to provide additional ways of accessing emergency features.

10. Improved Accessibility:

Accessibility features such as screen reader support, clear visual indicators, and adjustable text sizes can be introduced to improve usability for a wider range of users.

16. Conclusion

The SOS Safety System is a Flutter-based application developed to provide users with easy access to important safety-related features. It combines Firebase Authentication, Cloud Firestore, Geolocator, Provider, and other supporting technologies to create a cross-platform application.

The application includes features such as user registration, login, emergency contact management, SOS countdown, location access, safety tips, SOS history, and settings. These features are organized through a simple and user-friendly interface that supports both Android and Google Chrome platforms.

The project demonstrates the practical application of Flutter for cross-platform development and Firebase for authentication and cloud-based data management. It also provides experience in implementing device permissions, managing application state, designing responsive interfaces, and integrating external services.

Although the application has certain limitations, such as dependency on internet connectivity for some services and manual completion of emergency calls or messages, it provides a foundation for developing more advanced personal safety applications.

In conclusion, the SOS Safety System demonstrates how mobile and web technologies can be combined to develop an accessible safety application. Future enhancements can improve its reliability, security, and emergency communication capabilities.

17. References

The following documentation and online resources were referred to during the design and development of the SOS Safety System.

1. Flutter Documentation. *Flutter Official Documentation*. Available at: `https://docs.flutter.dev/`
2. Dart Documentation. *Dart Programming Language Documentation*. Available at: `https://dart.dev/guides`
3. Firebase Documentation. *Firebase Authentication*. Available at: `https://firebase.google.com/docs/auth`
4. Firebase Documentation. *Cloud Firestore*. Available at: `https://firebase.google.com/docs/firestore`

5. Firebase Documentation. *Firebase Cloud Messaging*. Available at: `https://firebase.google.com/docs/cloud-messaging`

6. Geolocator Package. *Geolocator Plugin for Flutter*. Available at: `https://pub.dev/packages/geolocator`

7. Provider Package. *Provider State Management for Flutter*. Available at: `https://pub.dev/packages/provider`

8. Shared Preferences Package. *Flutter Shared Preferences Plugin*. Available at: `https://pub.dev/packages/shared_preferences`

9. HTTP Package. *HTTP Package for Dart and Flutter*. Available at: `https://pub.dev/packages/http`

10. FlutterFire Documentation. *FlutterFire Documentation*. Available at: `https://firebase.flutter.dev/`